

FlowDesk Data Modeling Intern Assignment

Overview

FlowDesk is an internal document approval workflow product. The core challenge is not only storing documents and reviewers, but preserving the reasoning and evidence behind every state change. My design therefore separates:

- workflow configuration from workflow instances,
- document metadata from document file versions,
- reviewer assignments from reviewer actions,
- current state from immutable audit history.

The schema below assumes a PostgreSQL-style relational database.

Part 1: Requirements Analysis

Questions for the Product Manager

1. ****Are workflows configured globally, per organization, per team, per document type, or per individual document?****

This changes whether workflow templates need ownership fields, document type rules, or per-document overrides.

2. ****Can the same document type have different approval workflows depending on submitter, department, amount, risk level, or metadata?****

If routing is conditional, the schema needs workflow selection rules instead of a simple manual template choice.

3. ****Can admins edit a workflow after documents have already been submitted under it?****

This determines whether document workflows must snapshot the template at submission time to protect historical accuracy.

4. ****When a reviewer rejects a document, does the workflow restart from reviewer 1, return only to the rejecting reviewer, or continue after revision from a specific step?****

Rejection and resubmission behavior affects assignment state, document versioning, and active reviewer calculation.

5. ****Is a revision a new document, a new version of the same document, or just an update to the original file?****

Full history requires a document version model if revisions must be compared or audited.

6. ****Can a reviewer leave comments, attach evidence, or request changes without formally rejecting?****

This changes whether review actions need comments, attachments, and action types beyond approve/reject.

7. ****Can a reviewer be changed, delegated, skipped, or replaced after a workflow has started?****

This affects whether reviewer assignments are immutable or whether reassignment events must be modeled.

8. ****Can the same person be both submitter and reviewer?****

The answer affects validation rules and whether self-approval constraints are required.

9. ****Do reviewers need deadlines, reminders, or escalation rules?****

SLA fields would belong on workflow steps or assignments and would create additional events.

10. ****What roles exist besides submitter, reviewer, and admin?****

Permissions influence the user/role model and access control checks in APIs.

11. ****Should users belong to one organization only, or can they participate in multiple organizations or teams?****

Multi-organization membership requires a join table instead of a single `organization\_id` on users.

12. ****What does "full history" include: only approval actions, or also submission, file upload, edits, comments, reassignment, admin changes, and notification events?****

This defines the scope and granularity of the audit trail.

13. ****Do documents contain only metadata, or do we store actual files in FlowDesk?****

If files are stored externally, the database should store file metadata and object storage URIs instead of binary content.

14. ****Can workflows require all reviewers to approve, any one reviewer to approve, or a quorum of reviewers?****

This matters especially for parallel approval and may require approval policy fields.

15. ****Are approvals legally or compliance sensitive, requiring immutable audit logs and retention policies?****

This affects whether audit events can ever be edited or deleted and whether timestamps/users/IP addresses are required.

Assumptions

1. I am assuming FlowDesk is multi-tenant because the product description says "organizations manage workflows."
 2. I am assuming a user belongs to one organization for this first design because multi-organization membership was not specified.
 3. I am assuming document files are stored in object storage and the database stores metadata plus a `file\_uri`.
 4. I am assuming each resubmission creates a new `document\_versions` row so history remains clear.
 5. I am assuming workflow templates are configured by admins and copied into document workflow instances when a document is submitted.
 6. I am assuming a workflow can have one or more stages, with sequential approval represented by increasing stage numbers.
 7. I am assuming Part 2 starts with sequential approval, but I intentionally keep `stage\_number` in the schema because it also adapts cleanly to the parallel approval curveball.
 8. I am assuming a rejection sends the document back to the submitter and pauses the workflow until a new version is submitted.
 9. I am assuming only the active reviewer or active reviewers may take approval actions.
 10. I am assuming audit history is append-only and should not be edited through normal application flows.
 11. I am assuming admins can update workflow templates for future documents, but existing in-flight documents keep their original workflow assignments.
 12. I am assuming approval requires all assigned reviewers in the required stage or stages to approve unless the workflow policy says otherwise.
-

Part 2: Database Schema Design

Entity Summary

- `organizations`: tenant boundary for users, documents, and workflow templates.
- `users`: people who submit, review, or administer workflows.
- `documents`: current document-level state and ownership.
- `document\_versions`: every uploaded version of a document.
- `workflow\_templates`: reusable admin-configured approval workflow definitions.
- `workflow\_steps`: configured reviewer stages inside a workflow template.
- `document\_workflows`: a workflow instance attached to a submitted document.
- `document\_review\_assignments`: actual reviewer work items for a document workflow.
- `review\_actions`: reviewer decisions such as approve or reject.
- `audit\_events`: append-only timeline of important document and workflow events.

Tables

``organizations``

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Organization identifier
`name`	VARCHAR(200)	Not null	Organization name
`created_at`	TIMESTAMPTZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPTZ	Not null, default now()	Last update timestamp

`users`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	User identifier
`organization_id`	UUID	Not null, FK to `organizations(id)`	Tenant ownership
`email`	VARCHAR(255)	Not null	Login/contact email
`full_name`	VARCHAR(200)	Not null	Display name
`role`	VARCHAR(30)	Not null, check in (`admin`, `submitter`, `reviewer`)	Simplified role model
`is_active`	BOOLEAN	Not null, default true	Soft deactivation
`created_at`	TIMESTAMPTZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPTZ	Not null, default now()	Last update timestamp

Constraints:

- Unique: (`organization\_id`, `email`)

`documents`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Document identifier
`organization_id`	UUID	Not null, FK to `organizations(id)`	Tenant ownership
`submitter_id`	UUID	Not null, FK to `users(id)`	Original submitter
`current_version_id`	UUID	Nullable, FK to `document_versions(id)`	Current file version
`title`	VARCHAR(255)	Not null	Human-readable title
`description`	TEXT	Nullable	Optional description
`status`	VARCHAR(30)	Not null, check in (`draft`, `submitted`, `in_review`, `rejected`, `approved`, `cancelled`)	Current status
`created_at`	TIMESTAMPTZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPTZ	Not null, default now()	Last update timestamp

Indexes:

- (`organization\_id`, `status`)

- (`submitter\_id`, `created\_at`)

`document\_versions`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Version identifier
`document_id`	UUID	Not null, FK to `documents(id)`	Parent document
`version_number`	INTEGER	Not null, check `version_number > 0`	Sequential version number
`file_name`	VARCHAR(255)	Not null	Original file name
`file_uri`	TEXT	Not null	Object storage URI
`mime_type`	VARCHAR(100)	Not null	File type
`file_size_bytes`	BIGINT	Not null, check `file_size_bytes > 0`	File size
`submitted_by_id`	UUID	Not null, FK to `users(id)`	Uploader
`change_summary`	TEXT	Nullable	Reason for revision
`created_at`	TIMESTAMPTZ	Not null, default now()	Upload timestamp

Constraints:

- Unique: (`document\_id`, `version\_number`)

`workflow\_templates`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Template identifier
`organization_id`	UUID	Not null, FK to `organizations(id)`	Tenant ownership
`name`	VARCHAR(200)	Not null	Template name
`description`	TEXT	Nullable	Template purpose
`approval_mode`	VARCHAR(30)	Not null, default `sequential`, check in (`sequential`, `parallel`)	Default approval mode
`is_active`	BOOLEAN	Not null, default true	Whether template can be selected
`created_by_id`	UUID	Not null, FK to `users(id)`	Admin creator
`created_at`	TIMESTAMPTZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPTZ	Not null, default now()	Last update timestamp

Constraints:

- Unique: (`organization\_id`, `name`)

`workflow\_steps`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Step identifier
`workflow_template_id`	UUID	Not null, FK to `workflow_templates(id)`	Parent template
`reviewer_id`	UUID	Not null, FK to `users(id)`	Configured reviewer
`stage_number`	INTEGER	Not null, check `stage_number > 0`	Sequence stage
`step_order`	INTEGER	Not null, check `step_order > 0`	Stable ordering within display/config
`is_required`	BOOLEAN	Not null, default true	Whether reviewer approval is mandatory
`due_in_hours`	INTEGER	Nullable, check `due_in_hours > 0`	Optional SLA
`created_at`	TIMESTAMPZ	Not null, default now()	Creation timestamp

Constraints:

- Unique: (`workflow\_template\_id`, `step\_order`)
- Unique: (`workflow\_template\_id`, `reviewer\_id`, `stage\_number`)

Why both `stage\_number` and `step\_order`?

- `stage\_number` controls approval progression.
- `step\_order` gives a stable display/config order.
- In sequential mode, each required reviewer usually has a different `stage\_number`.
- In parallel mode, several reviewers may share the same `stage\_number`.

`document\_workflows`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Workflow instance identifier
`document_id`	UUID	Not null, unique, FK to `documents(id)`	One active workflow instance per document
`workflow_template_id`	UUID	Not null, FK to `workflow_templates(id)`	Source template
`approval_mode`	VARCHAR(30)	Not null, check in (`sequential`, `parallel`)	Snapshot from template
`status`	VARCHAR(30)	Not null, check in (`not_started`, `in_review`, `rejected`, `approved`, `cancelled`)	Workflow state
`current_stage_number`	INTEGER	Not null, default 1	Active stage
`started_at`	TIMESTAMPZ	Nullable	Review start
`completed_at`	TIMESTAMPZ	Nullable	Approval/cancellation time
`created_at`	TIMESTAMPZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPZ	Not null, default now()	Last update timestamp

`document\_review\_assignments`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Assignment identifier
`document_workflow_id`	UUID	Not null, FK to `document_workflows(id)`	Workflow instance
`document_version_id`	UUID	Not null, FK to `document_versions(id)`	Version being reviewed
`reviewer_id`	UUID	Not null, FK to `users(id)`	Assigned reviewer
`stage_number`	INTEGER	Not null, check `stage_number > 0`	Approval stage
`step_order`	INTEGER	Not null, check `step_order > 0`	Display/order snapshot
`status`	VARCHAR(30)	Not null, check in (`pending`, `active`, `approved`, `rejected`, `skipped`, `cancelled`)	Assignment state
`activated_at`	TIMESTAMPZ	Nullable	When reviewer became active
`completed_at`	TIMESTAMPZ	Nullable	When reviewer acted
`due_at`	TIMESTAMPZ	Nullable	Optional SLA deadline
`created_at`	TIMESTAMPZ	Not null, default now()	Creation timestamp
`updated_at`	TIMESTAMPZ	Not null, default now()	Last update timestamp

Constraints:

- Unique: (`document\_workflow\_id`, `reviewer\_id`, `document\_version\_id`, `stage\_number`)
- Unique: (`document\_workflow\_id`, `step\_order`, `document\_version\_id`)

`review\_actions`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Action identifier
`assignment_id`	UUID	Not null, FK to `document_review_assignments(id)`	Assignment being acted on
`document_id`	UUID	Not null, FK to `documents(id)`	Denormalized for querying
`document_version_id`	UUID	Not null, FK to `document_versions(id)`	Version reviewed
`reviewer_id`	UUID	Not null, FK to `users(id)`	Actor
`action`	VARCHAR(30)	Not null, check in (`approve`, `reject`)	Decision
`comment`	TEXT	Nullable	Reviewer comment
`created_at`	TIMESTAMPZ	Not null, default now()	Action timestamp

Constraints:

- Unique: (`assignment\_id`)

A reviewer assignment receives one final approve/reject action. If comment-only actions are later needed, I would add a separate `review\_comments` table or expand the action model.

`audit\_events`

Column	Type	Constraints	Notes
`id`	UUID	Primary key	Event identifier
`organization_id`	UUID	Not null, FK to `organizations(id)`	Tenant ownership
`document_id`	UUID	Not null, FK to `documents(id)`	Related document
`document_version_id`	UUID	Nullable, FK to `document_versions(id)`	Related version, if any
`actor_id`	UUID	Nullable, FK to `users(id)`	User or null for system
`event_type`	VARCHAR(50)	Not null	Event name
`message`	TEXT	Not null	Human-readable summary
`metadata`	JSONB	Not null, default `{}`	Structured event details
`created_at`	TIMESTAMPTZ	Not null, default now()	Event timestamp

Indexes:

- (`document\_id`, `created\_at`)
- (`organization\_id`, `created\_at`)
- (`event\_type`, `created\_at`)

Example `event\_type` values:

- `document.submitted`
- `document.version\_created`
- `workflow.started`
- `review.assignment\_activated`
- `review.approved`
- `review.rejected`
- `workflow.approved`
- `workflow.rejected`
- `document.resubmitted`
- `document.cancelled`

Relationship Description

- One `organization` has many `users`, `documents`, and `workflow\_templates`.
- One `document` belongs to one `organization` and one submitter.
- One `document` has many `document\_versions`.
- One `workflow\_template` has many `workflow\_steps`.
- One `document` has one `document\_workflow`.
- One `document\_workflow` has many `document\_review\_assignments`.
- One `document\_review\_assignment` has zero or one final `review\_action`.
- One `document` has many `audit\_events`.

Written ER view:

```
organizations 1--* users
organizations 1--* documents
organizations 1--* workflow_templates

users 1--* documents as submitter
documents 1--* document_versions
workflow_templates 1--* workflow_steps

documents 1--1 document_workflows
```

```
workflow_templates 1--* document_workflows
document_workflows 1--* document_review_assignments
document_review_assignments 1--0..1 review_actions

documents 1--* audit_events
document_versions 1--* audit_events
users 1--* audit_events as actor
```

Why I Structured Reviewer Sequence This Way

I used `workflow\_steps` and `document\_review\_assignments` instead of reviewer columns like `reviewer\_1\_id`, `reviewer\_2\_id`, and `reviewer\_3\_id`. This makes the number of reviewers flexible and keeps ordering explicit.

Alternatives considered:

- **Fixed reviewer columns:** Simple at first, but breaks as soon as the number of reviewers changes.
- **JSON array of reviewers on the document:** Flexible but weak for constraints, querying, indexing, and per-reviewer state.
- **Separate reviewer assignment rows:** Slightly more tables, but best for relational integrity, auditability, and sequential/parallel workflows.

I chose separate rows because every reviewer has independent state: pending, active, approved, rejected, skipped, due date, activation time, and completion time.

Where Document History Is Stored

I store history in two layers:

1. `review\_actions` stores structured reviewer decisions.
2. `audit\_events` stores the full timeline across document submission, versioning, workflow activation, review decisions, rejection, resubmission, and completion.

Trade-offs:

- This design is more verbose than updating only `documents.status`, but it preserves full traceability.
- `audit\_events.metadata` is flexible, but overusing JSON can hide important relational data. Therefore major entities still have normalized tables.
- Audit events should be append-only. If an event is wrong, the safer pattern is to append a correction event rather than editing history.

Areas I Am Not Fully Confident About

The biggest uncertainty is the rejection/resubmission policy. If rejection always restarts the full workflow, the assignment state machine is simple. If rejection returns only to the rejecting reviewer, the model still works but activation logic becomes more complex. I would ask the product manager to define this before implementation.

I am also not fully confident about role modeling. A simple `role` field is fine for the assignment, but a real enterprise product may need many-to-many roles, teams, departments, and permission scopes.

Part 3: API Contract Design

1. Submit a New Document for Approval

HTTP method and path

POST /documents

Request body

```
{
  "title": "Vendor Contract - June 2026",
  "description": "Contract requiring legal and finance approval.",
  "workflow_template_id": "8f4b2a15-3c08-4e9e-84bb-07de3c2db624",
  "file": {
    "file_name": "vendor-contract.pdf",
    "file_uri": "s3://flowdesk-documents/org-123/vendor-contract.pdf",
    "mime_type": "application/pdf",
    "file_size_bytes": 284921
  }
}
```

```

    },
    "change_summary": "Initial submission"
  }
}

```

Required fields:

```

- `title`
- `workflow_template_id`
- `file.file_name`
- `file.file_uri`
- `file.mime_type`
- `file.file_size_bytes`

```

Optional fields:

```

- `description`
- `change_summary`

```

****Success response: `201 Created`****

```

{
  "document": {
    "id": "c0e42e6e-5e18-4932-8cc4-9851dd1a2d6f",
    "title": "Vendor Contract - June 2026",
    "description": "Contract requiring legal and finance approval.",
    "status": "in_review",
    "submitter_id": "04dbbf66-23fd-4e64-8b12-77f2a32e47f4",
    "current_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339",
    "created_at": "2026-06-09T14:20:00Z"
  },
  "workflow": {
    "id": "36e2c4ea-b132-48d6-a4f4-1ec2a7cc50fb",
    "workflow_template_id": "8f4b2a15-3c08-4e9e-84bb-07de3c2db624",
    "approval_mode": "sequential",
    "status": "in_review",
    "current_stage_number": 1
  },
  "active_assignments": [
    {
      "id": "a93f57d1-5ca2-4ad2-b911-638a033c12d6",
      "reviewer_id": "472bc851-b9cb-47d7-9a1f-695af7f52b76",
      "stage_number": 1,
      "status": "active",
      "due_at": null
    }
  ]
}

```

****Error responses****

`400 Bad Request`

```

{
  "error": {
    "code": "INVALID_DOCUMENT_FILE",
    "message": "file_size_bytes must be greater than 0."
  }
}

```

`404 Not Found`

```

{
  "error": {
    "code": "WORKFLOW_TEMPLATE_NOT_FOUND",
    "message": "The selected workflow template does not exist or is not available in this organization."
  }
}

```

`409 Conflict`

```

{
  "error": {
    "code": "WORKFLOW_TEMPLATE_INACTIVE",
    "message": "The selected workflow template is inactive and cannot be used for new submissions."
  }
}

```

****Design decision****

The submit endpoint creates the document, first version, workflow instance, reviewer assignments, and audit events in one transaction. This avoids a half-submitted document with no workflow or a workflow with no file version.

2. Reviewer Takes an Action on a Document

****HTTP method and path****

POST /documents/{document_id}/review-actions

****Request body****

```
{
  "assignment_id": "a93f57d1-5ca2-4ad2-b911-638a033c12d6",
  "action": "approve",
  "comment": "Reviewed and approved."
}
```

Required fields:

- `assignment\_id`
- `action`

Optional fields:

- `comment`

Valid `action` values:

- `approve`
- `reject`

****Success response: `200 OK`****

```
{
  "review_action": {
    "id": "efe01d7b-e3bb-44e6-942b-5458fb39cd81",
    "assignment_id": "a93f57d1-5ca2-4ad2-b911-638a033c12d6",
    "document_id": "c0e42e6e-5e18-4932-8cc4-9851dd1a2d6f",
    "document_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339",
    "reviewer_id": "472bc851-b9cb-47d7-9a1f-695af7f52b76",
    "action": "approve",
    "comment": "Reviewed and approved.",
    "created_at": "2026-06-09T14:30:00Z"
  },
  "document": {
    "id": "c0e42e6e-5e18-4932-8cc4-9851dd1a2d6f",
    "status": "in_review"
  },
  "workflow": {
    "id": "36e2c4ea-b132-48d6-a4f4-1ec2a7cc50fb",
    "status": "in_review",
    "current_stage_number": 2
  },
  "active_assignments": [
    {
      "id": "de15891a-bb4e-4f4e-9d6c-f83024f953f6",
      "reviewer_id": "a6c85790-8978-4e60-aef4-c109ca5504c3",
      "stage_number": 2,
      "status": "active"
    }
  ]
}
```

If the action rejects the document, the response would show `document.status = "rejected"` and `workflow.status = "rejected"`.

****Error responses****

`403 Forbidden`

```
{
  "error": {
    "code": "NOT_ASSIGNED_REVIEWER",
    "message": "Only the assigned active reviewer can act on this assignment."
  }
}
```


`409 Conflict`

```
{
  "error": {
    "code": "ASSIGNMENT_NOT_ACTIVE",
    "message": "This review assignment is not currently active."
  }
}
```

`409 Conflict`

```
{
  "error": {
    "code": "ACTION_ALREADY_TAKEN",
    "message": "A final review action has already been recorded for this assignment."
  }
}
```

****Design decision****

The endpoint uses `assignment\_id` instead of only `reviewer\_id` because a reviewer may review multiple documents or multiple versions over time. The assignment is the precise work item being completed.

3. Fetch Full History / Timeline of a Document

****HTTP method and path****

GET /documents/{document_id}/timeline

****Request body****

None.

Optional query parameters:

- `include\_metadata`: boolean, default `false`
- `event\_type`: string, optional filter

****Success response: `200 OK`****

```
{
  "document": {
    "id": "c0e42e6e-5e18-4932-8cc4-9851dd1a2d6f",
    "title": "Vendor Contract - June 2026",
    "status": "in_review",
    "current_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339"
  },
  "timeline": [
    {
      "id": "9ac72153-5b2b-4b9d-9e99-5646d8b405e8",
      "event_type": "document.submitted",
      "message": "Document submitted for approval.",
      "actor": {
        "id": "04dbbf66-23fd-4e64-8b12-77f2a32e47f4",
        "full_name": "Ananya Rao"
      },
      "document_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339",
      "created_at": "2026-06-09T14:20:00Z"
    },
    {
      "id": "6ea30b16-2fd5-4724-bd65-2c27fc59e0e1",
      "event_type": "review.assignment_activated",
      "message": "Reviewer Raj Mehta was assigned at stage 1.",
      "actor": null,
      "document_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339",
      "created_at": "2026-06-09T14:20:01Z"
    },
    {
      "id": "7811e186-5521-44df-9acf-096522b7e82c",
      "event_type": "review.approved",
      "message": "Raj Mehta approved version 1.",
      "actor": {
        "id": "472bc851-b9cb-47d7-9a1f-695af7f52b76",
        "full_name": "Raj Mehta"
      },
      "document_version_id": "b2cdd6d9-d331-4910-880f-7c47f3c84339",
      "created_at": "2026-06-09T14:30:00Z"
    }
  ]
}
```

****Error responses****

``404 Not Found``

```
{
  "error": {
    "code": "DOCUMENT_NOT_FOUND",
    "message": "Document not found."
  }
}
```

``403 Forbidden``

```
{
  "error": {
    "code": "DOCUMENT_ACCESS_DENIED",
    "message": "You do not have permission to view this document timeline."
  }
}
```

****Design decision****

The timeline is driven by ``audit_events`` instead of reconstructing everything from current row states. This ensures that events like reassignment, activation, rejection, and resubmission appear exactly as they occurred.

Part 4: Spot the Problems

Proposed schema:

`document_reviewers`

```
id INT (PK)
document_id INT
reviewer_1_id INT
reviewer_2_id INT
reviewer_3_id INT
reviewer_1_done BOOLEAN
reviewer_2_done BOOLEAN
reviewer_3_done BOOLEAN
all_approved BOOLEAN
```

Problems

1. ****It supports exactly three reviewers.****

If a workflow needs one, two, four, or ten reviewers, the table either wastes columns or requires a schema migration.

2. ****Reviewer order is hardcoded into column names.****

Queries and application logic must know about ``reviewer_1_id``, ``reviewer_2_id``, and ``reviewer_3_id`` separately instead of treating reviewer steps uniformly.

3. ****It cannot represent parallel approvals cleanly.****

There is no stage, group, or approval mode field to say whether reviewers act one at a time or together.

4. ****It stores "done" instead of actual decisions.****

A reviewer being done does not say whether they approved, rejected, skipped, or were reassigned.

5. ****`all\_approved` is derived state.****

It can become inconsistent with the reviewer flags. For example, ``all_approved = true`` while ``reviewer_2_done = false``.

6. ****There are no foreign key constraints shown.****

The database would not prevent invalid ``document_id`` or reviewer IDs.

7. ****There is no audit trail.****

The schema cannot answer who approved when, who rejected, what comment was left, or what version was reviewed.

8. ****There are no timestamps.****

The product requires full history, but the schema cannot show when reviewers acted.

9. ****There is no support for rejection reason or reviewer comments.****

Rejections are one of the core product actions, so losing the reason makes the workflow much less useful.

10. ****There is no document version relationship.****

If the submitter revises and resubmits, the schema cannot tell which version each reviewer approved or rejected.

11. ****It mixes workflow configuration and workflow state.****

Reviewer identities and reviewer completion flags are stored together, making it hard to reuse admin-configured workflows.

12. ****It is difficult to query.****

Finding all pending reviews for a user requires checking three different columns instead of filtering rows by `reviewer\_id` and `status`.

13. ****It cannot represent active versus pending reviewers.****

In sequential workflows, reviewer 2 may be assigned but not yet active. A boolean `done` flag does not capture this.

14. ****It cannot handle reviewer replacement or delegation.****

If reviewer 2 is replaced, the old assignment disappears unless overwritten, destroying history.

15. ****It has no tenant boundary.****

In an organization-based product, the schema should prevent cross-organization leakage.

What Breaks or Becomes Difficult

- Adding a fourth reviewer requires changing the database schema, backend logic, API response shape, and UI.
- Reporting pending work for a reviewer becomes awkward and slow.
- Rejection flows are under-specified because there is nowhere to store rejection details.
- Full history is impossible because old states are overwritten.
- Parallel approvals cannot be modeled without adding more columns or ambiguous booleans.
- Data integrity depends too much on application code instead of database constraints.

How My Schema Addresses These Problems

- `workflow\_steps` supports any number of configured reviewers.
 - `document\_review\_assignments` stores one row per reviewer per document version.
 - `stage\_number` models sequential order and can also group parallel reviewers.
 - `status` on assignments distinguishes pending, active, approved, rejected, skipped, and cancelled.
 - `review\_actions` stores the actual approve/reject decision with comments and timestamps.
 - `audit\_events` preserves the full timeline.
 - Foreign keys connect documents, versions, workflows, reviewers, and actions.
 - Unique constraints prevent duplicate reviewer assignments within the same workflow/version/stage.
 - Document versions make resubmission auditable.
-

Part 5: The Curveball - Parallel Approvals

New requirement:

> Sometimes all reviewers should review at the same time, and the document is approved only when all of them approve. Sometimes it should still be sequential. This should be configurable per workflow.

1. Does This Break or Complicate the Existing Design?

It complicates the design, but it does not fully break it because the schema already uses rows for reviewer assignments instead of fixed reviewer columns.

The important impact is workflow activation logic:

- In sequential mode, only reviewers in the current stage should be active.
- In parallel mode, all required reviewers may become active at the same time.
- Final approval in parallel mode requires checking that every required active assignment has approved.

If I had only modeled a strict `step\_order` with one active reviewer at a time, parallel approval would require a bigger redesign. Because the schema includes `stage\_number`, multiple reviewers can share the same stage.

2. Schema Changes to Support Parallel Approval

I would keep the existing tables but make the following changes explicit:

1. Add or confirm `workflow\_templates.approval\_mode`:

```
approval_mode VARCHAR(30)
CHECK approval_mode IN ('sequential', 'parallel')
```

2. Add or confirm `document\_workflows.approval\_mode` as a snapshot from the template:

```
approval_mode VARCHAR(30)
CHECK approval_mode IN ('sequential', 'parallel')
```

3. Use `workflow\_steps.stage\_number` to group reviewers:

- Sequential example: reviewer A stage 1, reviewer B stage 2, reviewer C stage 3.
- Parallel example: reviewer A stage 1, reviewer B stage 1, reviewer C stage 1.

4. Add `approval\_policy` later if the product expands beyond "all reviewers must approve":

```
approval_policy VARCHAR(30)
CHECK approval_policy IN ('all_required', 'any_one', 'quorum')
```

I would not add this policy in the first implementation unless the PM confirms quorum or any-one approval is needed.

3. API Contract Changes

Submit Document API

The `POST /documents` response should expose the workflow approval mode and all active assignments.

Sequential response may include one active assignment:

```
{
  "workflow": {
    "approval_mode": "sequential",
    "current_stage_number": 1
  },
  "active_assignments": [
    {
      "reviewer_id": "reviewer-a",
      "stage_number": 1,
      "status": "active"
    }
  ]
}
```

Parallel response may include several active assignments:

```
{
  "workflow": {
    "approval_mode": "parallel",
    "current_stage_number": 1
  },
  "active_assignments": [
    {
      "reviewer_id": "reviewer-a",
      "stage_number": 1,
      "status": "active"
    },
    {
      "reviewer_id": "reviewer-b",
      "stage_number": 1,
      "status": "active"
    }
  ]
}
```

Review Action API

The `POST /documents/{document\_id}/review-actions` response should include whether the document is still waiting on other reviewers.

```
{
  "review_action": {
    "action": "approve",
    "reviewer_id": "reviewer-a"
  },
  "workflow": {
    "approval_mode": "parallel",
    "status": "in_review",
    "current_stage_number": 1,
    "remaining_required_approvals": 1
  }
}
```

When the final required reviewer approves, the workflow and document become approved.

Timeline API

The timeline should expose stage and mode metadata so the reader can understand why multiple reviewers were active at once.

```
{
  "event_type": "review.assignment_activated",
  "message": "Parallel review stage 1 activated for 3 reviewers.",
  "metadata": {
    "approval_mode": "parallel",
    "stage_number": 1,
    "assignment_count": 3
  }
}
```

4. Remaining Ambiguities

1. In parallel mode, does one rejection immediately reject the document, or do other reviewers still complete their reviews?
 2. Is approval always "all reviewers approve," or can a workflow require a quorum such as 2 of 3?
 3. Can a workflow mix sequential and parallel stages, such as legal and finance in parallel, then CEO approval after both finish?
 4. Can reviewers be added or removed after a parallel workflow starts?
 5. Should the submitter receive partial feedback as reviewers respond, or only after the whole stage completes?
 6. If one parallel reviewer rejects and another approves, what status should each assignment keep after resubmission?
-

Validation Scenarios

1. ****Single reviewer approval****
 - One workflow step creates one assignment.
 - Reviewer approves.
 - Document becomes `approved`.
 - Timeline shows submission, activation, approval, workflow approval.
2. ****Three sequential reviewers****
 - Three workflow steps with stages 1, 2, and 3.
 - Only stage 1 starts active.
 - Each approval activates the next stage.
 - Final approval marks document approved.
3. ****Rejection and resubmission****
 - Active reviewer rejects with a comment.

- Document becomes `rejected`.
 - Submitter uploads version 2.
 - New assignments reference version 2.
 - Timeline preserves version 1 rejection and version 2 resubmission.
4. **Full audit timeline**
- Fetching `GET /documents/{document\_id}/timeline` returns ordered events across versions and review actions.
5. **Parallel approvals**
- Several reviewers share stage 1.
 - All assignments start active.
 - Document remains `in\_review` until all required reviewers approve.
6. **Parallel rejection**
- One reviewer rejects.
 - Based on the current assumption, the document becomes `rejected` immediately.
 - The exact product behavior should be confirmed with the PM.
7. **Admin updates workflow template**
- Existing document workflow keeps its assignment snapshot.
 - New documents use the updated template.
-

Tooling and Framework Note

If this assignment were extended into a runnable prototype, I would choose **Django + Django REST Framework** because FlowDesk is centered on relational data, admin-managed workflow configuration, authentication, permissions, and REST APIs. Django models would map naturally to the tables above, and Django Admin would be useful for workflow template management.

Other options:

- **FastAPI + SQLAlchemy:** strong API-first option, but admin and permissions require more manual setup.
- **Reflex:** good for a pure-Python visual prototype, but not the strongest choice for showing backend data-modeling depth.
- **Streamlit:** fastest for a lightweight demo, but not ideal for a real approval workflow application.

I would avoid vector databases, web scrapers, and small local language models for the core assignment because the assignment is about relational modeling and reasoning. AI tools may be useful only as reviewers to identify missing assumptions or unclear API behavior.

Security note: any API keys accidentally shared during development should be rotated and replaced with new values stored only in environment variables such as `.env`, never committed to the submission or code.